

Machine Learning & AI — Complete Reference (Fundamentals to Frontier Models)

A structured, code-and-formula-backed path through the algorithms and architectures that make up the modern ML/AI stack. Every entry includes a **definition**, **purpose**, **use cases**, the **core formula** (where mathematically defined), and a **runnable code snippet**.

Contents

WHAT THIS GUIDE COVERS

→ Suggested Learning Order

→ 1. Machine Learning Fundamentals

→ 2. Classification Algorithms

→ 3. Clustering

→ 4. Dimensionality Reduction

→ 5. Association Rule Mining

→ 6. Reinforcement Learning

→ 7. Neural Networks

→ 8. Convolutional Neural Networks (CNN)

→ 9. Object Detection

→ 10. Semantic Segmentation

→ 11. Recurrent Neural Networks

→ 12. Attention Models

→ 13. Transformers

→ 14. Generative Models

→ 15. Graph Neural Networks

→ 16. Time-Series Forecasting

→ 17. Recommendation Systems

→ 18. Anomaly Detection

→ 19. Ensemble Learning

→ 20. Optimisation Algorithms (Deep Learning)

→ 21. Loss Functions

Suggested Learning Order

1. Python, NumPy, Pandas, Matplotlib
2. Linear & Logistic Regression
3. KNN, Decision Trees, SVM, Naive Bayes
4. Random Forest, Gradient Boosting, XGBoost, LightGBM, CatBoost
5. Clustering (K-Means, DBSCAN, GMM)
6. PCA, t-SNE, UMAP
7. Neural Networks (Perceptron, MLP)
8. CNNs and transfer learning
9. RNN, LSTM, GRU
0. Attention and Transformers
 1. Large Language Models (BERT, GPT, LLaMA, Qwen, Gemma)
 2. Reinforcement Learning
 3. GANs and Diffusion Models
 4. Graph Neural Networks
 5. Time-Series Forecasting
 6. MLOps, distributed training, model deployment, and inference optimisation

This sequence moves from fundamentals to the state-of-the-art systems used in modern AI production.

Supervised algorithms that predict a **continuous** numeric output from input features.

• Linear Regression

Definition: Models the relationship between features and a continuous target as a straight line (or hyperplane).

Purpose: predict continuous values

Use Cases: house price prediction, salary prediction, stock trend estimation, sales forecasting

Formula:

FORMULA

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n \quad (y = mx + c)$$
$$\text{Loss (MSE)} = (1/n) \sum (y_i - \hat{y}_i)^2$$

Code:

PYTHON

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
```

• Polynomial Regression

Definition: Extends linear regression with polynomial terms of the features to fit curved relationships.

Purpose: capture non-linear trends using a linear model on transformed features

Use Cases: curve fitting, growth prediction, engineering analysis

Formula:

FORMULA

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_n x^n$$

Code:

PYTHON

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

model = LinearRegression()
model.fit(X_poly, y)
```

• Ridge Regression

Definition: Linear regression with an L2 penalty that shrinks coefficients toward zero.

Purpose: L2 regularization to reduce overfitting **Use Cases:** reduce overfitting, high-dimensional datasets **Formula:**

FORMULA

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$$

Code:

PYTHON

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0)
model.fit(X_train, y_train)
```

• Lasso Regression

Definition: Linear regression with an L1 penalty that can shrink coefficients exactly to zero, giving automatic feature selection.

Purpose: feature selection and sparse models

Use Cases: feature selection, sparse models

Formula:

FORMULA

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j|$$

Code:

PYTHON

```
from sklearn.linear_model import Lasso

model = Lasso(alpha=0.1)
model.fit(X_train, y_train)
```

• ElasticNet

Definition: Combines L1 and L2 penalties, blending Ridge and Lasso.

Purpose: balance feature selection with coefficient shrinkage

Use Cases: combination of Ridge + Lasso use cases

Formula:

FORMULA

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \lambda_1 \sum |\beta_j| + \lambda_2 \sum \beta_j^2$$

Code:

PYTHON

```
from sklearn.linear_model import ElasticNet

model = ElasticNet(alpha=0.1, l1_ratio=0.5)
model.fit(X_train, y_train)
```

02 Classification Algorithms

Supervised algorithms that predict a **discrete category/class**.

• Logistic Regression

Definition: A linear model that applies the sigmoid function to output class probabilities.

Purpose: probabilistic linear classification

Use Cases: spam detection, disease prediction, credit approval

Formula:

FORMULA

$$P(y=1|x) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)})$$
$$\text{Loss (BCE)} = -(1/n) \sum [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

Code:

PYTHON

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
```

• Naive Bayes

Definition: A probabilistic classifier based on Bayes' theorem with a feature-independence assumption.

Variants: Gaussian NB, Multinomial NB, Bernoulli NB

Purpose: fast probabilistic classification, especially for text

Use Cases: email spam, NLP, document classification

Formula:

FORMULA

$$P(\text{class}|\text{features}) = [P(\text{features}|\text{class}) \cdot P(\text{class})] / P(\text{features})$$

Code:

PYTHON

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)
```

• K-Nearest Neighbours (KNN)

Definition: A non-parametric algorithm that classifies a point by majority vote among its k nearest neighbors.

Purpose: similarity-based classification

Use Cases: recommendation, image classification, medical diagnosis

Formula:

FORMULA

$$\text{Euclidean distance: } d(x,y) = \sqrt{[\sum (x_i - y_i)^2]}$$

Code:

PYTHON

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
```


• Support Vector Machine (SVM)

Definition: Finds the hyperplane that maximizes the margin between classes; kernels enable non-linear boundaries.

Purpose: margin-based classification, strong in high dimensions

Use Cases: face recognition, text classification, bioinformatics

Formula:

FORMULA

Decision: $f(x) = \text{sign}(w^T x + b)$

Objective: minimize $\frac{1}{2} \|w\|^2$ s.t. $y_i (w^T x_i + b) \geq 1$

Code:

PYTHON

```
from sklearn.svm import SVC

model = SVC(kernel="rbf")
model.fit(X_train, y_train)
```

• Decision Tree

Definition: A tree that recursively splits data on the feature/threshold that best separates classes.

Purpose: interpretable, rule-based classification/regression

Use Cases: loan approval, fraud detection, customer segmentation

Formula:

FORMULA

Gini Impurity = $1 - \sum p_i^2$

Entropy = $-\sum p_i \log_2(p_i)$

Code:

PYTHON

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
model.fit(X_train, y_train)
```

• Random Forest

Definition: An ensemble (bagging) of decision trees trained on bootstrapped samples and random feature subsets, combined by majority vote.

Purpose: reduce the variance/overfitting of a single tree

Use Cases: medical diagnosis, stock prediction, feature importance

Formula:

FORMULA

$$\hat{y} = \text{majority_vote}(\text{Tree}_1(x), \text{Tree}_2(x), \dots, \text{Tree}_n(x))$$

Code:

PYTHON

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X_train, y_train)
```

• Extra Trees

Definition: Like Random Forest, but splits use random thresholds instead of the best threshold — more randomness, faster training.

Purpose: faster, lower-variance alternative to Random Forest

Use Cases: faster Random Forest, high-dimensional data

Code:

PYTHON

```
from sklearn.ensemble import ExtraTreesClassifier

model = ExtraTreesClassifier()
model.fit(X_train, y_train)
```

• Gradient Boosting

Definition: Builds trees sequentially, each correcting the residual errors of the previous ensemble via gradient descent on the loss.

Purpose: high-accuracy sequential ensembling

Use Cases: predictive analytics, insurance, banking

Formula:

FORMULA

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x) \quad (h_m \text{ fits the negative gradient of the loss})$$

Code:

PYTHON

```
from sklearn.ensemble import GradientBoostingClassifier

model = GradientBoostingClassifier()
model.fit(X_train, y_train)
```

• AdaBoost

Definition: Adaptive boosting that reweights misclassified samples so later weak learners focus on them.

Purpose: turn many weak learners into one strong classifier

Use Cases: face detection, object detection

Formula:

FORMULA

$$w_i \leftarrow w_i \cdot \exp(\alpha \cdot [y_i \neq h_m(x_i)]) \quad \alpha = 0.5 \cdot \ln((1-\text{err})/\text{err})$$

Code:

PYTHON

```
from sklearn.ensemble import AdaBoostClassifier

model = AdaBoostClassifier()
model.fit(X_train, y_train)
```

• XGBoost

Definition: A regularized, second-order-gradient implementation of gradient boosting, optimized for speed and sparse data.

Purpose: highly efficient regularized boosting

Use Cases: Kaggle competitions, finance, recommendation

Code:

PYTHON

```
from xgboost import XGBClassifier

model = XGBClassifier()
model.fit(X_train, y_train)
```

• LightGBM

Definition: A histogram-based, leaf-wise gradient boosting framework built for speed on large datasets.

Purpose: fast, memory-efficient boosting at scale

Use Cases: large datasets, production ML

Code:

PYTHON

```
from lightgbm import LGBMClassifier

model = LGBMClassifier()
model.fit(X_train, y_train)
```

- **CatBoost**

Definition: A gradient boosting library with native, efficient handling of categorical features.

Purpose: boosting optimized for categorical data

Use Cases: categorical data

Code:

PYTHON

```
from catboost import CatBoostClassifier

model = CatBoostClassifier(verbose=0)
model.fit(X_train, y_train)
```

03 Clustering

Unsupervised algorithms that group similar data points together.

• K-Means

Definition: Partitions data into k clusters by iteratively assigning points to the nearest centroid and recomputing centroids.

Purpose: distance-based unsupervised partitioning

Use Cases: customer segmentation, image compression

Formula:

FORMULA

Objective (WCSS): minimize $\sum_k \sum_{(x \in C_k)} \|x - \mu_k\|^2$

Code:

PYTHON

```
from sklearn.cluster import KMeans

model = KMeans(n_clusters=3)
model.fit(X)
```

• DBSCAN

Definition: Density-based clustering that groups closely packed points and flags low-density points as noise.

Purpose: discover arbitrarily-shaped clusters and detect outliers

Use Cases: noise detection, GPS data

Formula:

FORMULA

Core point if: $|\{q : \text{dist}(p,q) \leq \epsilon\}| \geq \text{minPts}$

Code:

PYTHON

```
from sklearn.cluster import DBSCAN

model = DBSCAN(eps=0.5, min_samples=5)
model.fit(X)
```

• Hierarchical Clustering

Definition: Builds a dendrogram of nested clusters by iteratively merging (agglomerative) or splitting (divisive) groups.

Purpose: build nested clusters without pre-specifying k

Use Cases: biology, gene analysis

Formula:

FORMULA

Average linkage: $d(A,B) = (1/(|A||B|)) \sum_{i \in A} \sum_{j \in B} \text{dist}(i,j)$

Code:

PYTHON

```
from sklearn.cluster import AgglomerativeClustering

model = AgglomerativeClustering(n_clusters=3)
model.fit(X)
```

- **Gaussian Mixture Model**

Definition: Models data as a mixture of Gaussian distributions, assigning soft (probabilistic) cluster membership via Expectation-Maximization.

Purpose: soft clustering and density estimation

Use Cases: soft clustering, density estimation

Formula:

FORMULA

$$p(x) = \sum_k \pi_k \cdot N(x \mid \mu_k, \Sigma_k)$$

Code:

PYTHON

```
from sklearn.mixture import GaussianMixture

model = GaussianMixture(n_components=3)
model.fit(X)
```


04 Dimensionality Reduction

Techniques that compress features while retaining the most useful structure.

• PCA

Definition: Projects data onto orthogonal directions of maximum variance, derived from the eigenvectors of the covariance matrix.

Purpose: reduce dimensionality while preserving variance

Use Cases: data visualization, feature reduction

Formula:

FORMULA

Covariance: $\Sigma = (1/n) X^T X$

Principal components = eigenvectors of Σ , ranked by eigenvalue

Code:

PYTHON

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)
```

• Kernel PCA

Definition: Applies the kernel trick to PCA to capture non-linear structure by implicitly mapping data to higher dimensions.

Purpose: non-linear dimensionality reduction

Use Cases: nonlinear reduction

Formula:

FORMULA

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j) \quad \text{e.g. RBF: } K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Code:

PYTHON

```
from sklearn.decomposition import KernelPCA

kpca = KernelPCA(kernel="rbf")
X_reduced = kpca.fit_transform(X)
```

• t-SNE

Definition: Converts pairwise similarities into probabilities and minimizes their KL divergence in low dimensions — built for visualization.

Purpose: visualize high-dimensional data in 2D/3D

Use Cases: visualization

Formula:

FORMULA

$$\text{Minimize: } KL(P||Q) = \sum_{i,j} p_{ij} \log(p_{ij}/q_{ij})$$

Code:

PYTHON

```
from sklearn.manifold import TSNE

model = TSNE(n_components=2)
X_embedded = model.fit_transform(X)
```

• UMAP

Definition: Builds a fuzzy topological graph of the data and optimizes a low-dimensional layout to match it — faster than t-SNE with better global structure preservation.

Purpose: scalable non-linear dimensionality reduction

Use Cases: large-dataset visualization

Code:

PYTHON

```
import umap

model = umap.UMAP()
X_embedded = model.fit_transform(X)
```

05 Association Rule Mining

Definition: Unsupervised techniques that discover "if X then Y" relationships between items in transactional data.

Purpose: find frequent co-occurrence patterns in transactions

Use Cases: market basket analysis, product recommendation

Algorithms: Apriori, FP-Growth, ECLAT

Formula:

FORMULA

```
Support(X)      = count(X) / N
Confidence(X→Y) = Support(XuY) / Support(X)
Lift(X→Y)       = Confidence(X→Y) / Support(Y)
```

Code:

PYTHON

```
from mlxtend.frequent_patterns import apriori, association_rules

frequent_items = apriori(df, min_support=0.05, use_colnames=True)
rules = association_rules(frequent_items, metric="lift", min_threshold=1.0)
```

Definition: An agent learns to act in an environment to maximize cumulative reward through trial and error.

Purpose: sequential decision-making under uncertainty

Use Cases: robotics, autonomous driving, games, trading

Algorithms: Q-Learning, SARSA, Deep Q-Network (DQN), Double DQN, Dueling DQN, PPO, A2C, A3C, DDPG, TD3, SAC

Formula (Bellman Equation):

FORMULA

$$Q(s,a) = r + \gamma \cdot \max_{a'} Q(s',a')$$

Formula (Q-Learning update — off-policy):

PYTHON

```
Q[state, action] += alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state, action]
)
```

Formula (SARSA update — on-policy):

FORMULA

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$$

Formula (PPO clipped objective):

FORMULA

$$L(\theta) = E[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$$

Code:

PYTHON

```
import gymnasium as gym
from stable_baselines3 import PPO

env = gym.make("CartPole-v1")
model = PPO("MlpPolicy", env, verbose=0)
model.learn(total_timesteps=10000)
```

Definition: Computational models loosely inspired by biological neurons — layers of weighted connections passed through non-linear activation functions.

Purpose: learn complex, non-linear mappings from data

- **Perceptron**

Definition: The simplest neural unit — a weighted sum of inputs passed through a step/threshold activation.

Purpose: binary linear classification

Use Cases: binary classification

Formula:

FORMULA

$\hat{y} = \text{step}(w \cdot x + b)$
Update rule: $w \leftarrow w + \alpha(y - \hat{y})x$

Code:

PYTHON

```
import torch.nn as nn

model = nn.Linear(10, 1)
```

- **Multi-Layer Perceptron (MLP)**

Definition: A feedforward network with one or more hidden layers and non-linear activations, trained via backpropagation.

Purpose: general-purpose function approximation

Use Cases: classification, regression

Formula:

FORMULA

$h = \text{activation}(W_1x + b_1)$

$\hat{y} = \text{activation}(W_2h + b_2)$

Activations: $\text{sigmoid}(x)=1/(1+e^{-x})$, $\text{ReLU}(x)=\max(0,x)$, $\tanh(x)$

Code:

PYTHON

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 10)
)
```

08 Convolutional Neural Networks (CNN)

Definition: Neural networks that use learnable convolutional filters to extract spatial, hierarchical features from grid-like data (e.g. images).

Purpose: automatic feature extraction from images

Use Cases: image classification, medical imaging, face recognition, OCR

Formula:

FORMULA

Convolution: $(I * K)(i,j) = \sum \sum I(i+m, j+n) \cdot K(m,n)$
Output size: $O = \lfloor (W - F + 2P) / S \rfloor + 1$ (W=input, F=filter, P=padding, S=stride)

Popular Architectures:

Model	Year	Key Idea
LeNet	1998	First practical CNN, digit recognition
AlexNet	2012	Deep CNN + ReLU + dropout, ImageNet breakthrough
VGG16 / VGG19	2014	Deep stacks of small 3×3 convolutions
GoogLeNet (Inception)	2014	Multi-scale "Inception" modules
ResNet	2015	Residual/skip connections enabling very deep nets
ResNeXt	2017	Grouped convolutions ("cardinality")
DenseNet	2017	Every layer connected to all previous layers
MobileNet	2017	Depthwise-separable convolutions for mobile
ShuffleNet	2018	Channel shuffle + group convolutions
EfficientNet	2019	Compound scaling of depth/width/resolution
ConvNeXt	2022	Modernized CNN design inspired by Transformers

Code:

PYTHON

```
import torch.nn as nn
import torchvision.models as models

# Build a small CNN from scratch
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(32 * 16 * 16, 10)

    def forward(self, x):
        x = self.pool(nn.functional.relu(self.conv1(x)))
        return self.fc(x.flatten(1))

# Or use a pretrained architecture
resnet = models.resnet50(weights="IMAGENET1K_V2")
```

09 Object Detection

Definition: Localizes and classifies multiple objects in an image by predicting bounding boxes and class labels.

Purpose: identify "what" and "where" objects are

Use Cases: autonomous cars, CCTV, traffic monitoring

Algorithms: R-CNN, Fast R-CNN, Faster R-CNN, Mask R-CNN, SSD, YOLOv1-YOLOv12, RetinaNet, DETR, RT-DETR

Formula (Intersection over Union):

FORMULA

$$\text{IoU} = \text{Area}(\text{Prediction} \cap \text{Ground Truth}) / \text{Area}(\text{Prediction} \cup \text{Ground Truth})$$

Formula (mean Average Precision):

FORMULA

$$\text{mAP} = (1/N) \sum AP_i \quad (\text{AP} = \text{area under the Precision-Recall curve for class } i)$$

Family	Key Idea
R-CNN / Fast / Faster R-CNN	Region proposals + CNN classifier, two-stage detector
Mask R-CNN	Faster R-CNN + pixel-level instance segmentation branch
SSD	Single-shot, multi-scale anchor boxes
YOLO (v1-v12)	Single-shot, grid-based, real-time detection
RetinaNet	Single-shot + Focal Loss to handle class imbalance
DETR / RT-DETR	Transformer-based set-prediction detection (no anchors/NMS)

Code:

PYTHON

```
# Faster R-CNN (torchvision)
import torchvision
model = torchvision.models.detection.fasterrcnn_resnet50_fpn(weights="DEFAULT")

# YOLO (ultralytics)
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
results = model("image.jpg")
```

Definition: Classifies every pixel in an image into a category, producing a dense per-pixel label map.

Purpose: pixel-level scene understanding

Use Cases: medical imaging, satellite images

Algorithms: FCN, U-Net, DeepLab, PSPNet, SegFormer, Mask2Former

Formula (Dice Coefficient):

FORMULA

$$\text{Dice} = 2|A \cap B| / (|A| + |B|)$$

Formula (IoU / Jaccard Index):

FORMULA

$$\text{IoU} = |A \cap B| / |A \cup B|$$

Code:

PYTHON

```
import segmentation_models_pytorch as smp

model = smp.Unet(
    encoder_name="resnet34",
    encoder_weights="imagenet",
    in_channels=3,
    classes=1
)
```

Definition: Networks with recurrent connections that carry a hidden state across time steps, suited to sequential data.

Purpose: model sequences and temporal dependencies

Use Cases: NLP, time series, speech recognition

Algorithms: RNN, LSTM, GRU, BiLSTM, Seq2Seq

Formula (Vanilla RNN):

FORMULA

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Formula (LSTM gates):

FORMULA

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) && \# \text{ forget gate} \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) && \# \text{ input gate} \\ o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) && \# \text{ output gate} \\ C_t &= f_t * C_{t-1} + i_t * \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\ h_t &= o_t * \tanh(C_t) \end{aligned}$$

Formula (GRU gates):

FORMULA

$$\begin{aligned} z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) && \# \text{ update gate} \\ r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) && \# \text{ reset gate} \\ h_t &= (1 - z_t) * h_{t-1} + z_t * \tanh(W \cdot [r_t * h_{t-1}, x_t]) \end{aligned}$$

Code:

PYTHON

```
import torch.nn as nn

lstm = nn.LSTM(input_size=10, hidden_size=64, num_layers=2, batch_first=True)
gru = nn.GRU(input_size=10, hidden_size=64, batch_first=True)
```

12

Attention Models

Definition: Mechanisms that let a model dynamically weigh which parts of the input matter most when producing each output.

Purpose: capture long-range dependencies without recurrence

Use Cases: translation, LLMs

Algorithms: Bahdanau Attention, Luong Attention, Self-Attention, Multi-Head Attention

Formula (Scaled Dot-Product Attention):

FORMULA

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Formula (Multi-Head Attention):

FORMULA

$$\begin{aligned} \text{MultiHead}(Q,K,V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \\ \text{head}_i &= \text{Attention}(QW_iQ, KW_iK, VW_iV) \end{aligned}$$

Code:

PYTHON

```
import torch.nn as nn

attn = nn.MultiheadAttention(embed_dim=512, num_heads=8, batch_first=True)
output, weights = attn(query, key, value)
```

13 Transformers

Definition: Architectures built entirely from self-attention and feedforward layers (no recurrence), enabling massive parallelism and scale.

Purpose: sequence modeling at scale — the backbone of modern LLMs

Use Cases: chatbots, search, code generation, translation

Popular Models: Transformer, BERT, RoBERTa, ALBERT, DistilBERT, GPT/GPT-2/GPT-3/GPT-4/GPT-5, T5, BART, ELECTRA, XLNet, LLaMA, Mistral, Gemma, Qwen, DeepSeek, Phi

Formula (Positional Encoding):

FORMULA

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d)}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d)}) \end{aligned}$$

Code:

PYTHON

```
import torch.nn as nn

layer = nn.TransformerEncoderLayer(d_model=512, nhead=8)
encoder = nn.TransformerEncoder(layer, num_layers=6)

# Using a pretrained model
from transformers import AutoModel, AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")
```

14 Generative Models

Definition: Models that learn the underlying data distribution in order to generate new, realistic samples.

Purpose: synthesize new data resembling the training distribution

Use Cases: image generation, video generation, audio generation, image restoration

Algorithms: Autoencoder, Variational Autoencoder (VAE), GAN, DCGAN, CycleGAN, StyleGAN, Diffusion Models, Stable Diffusion, DDPM, DDIM, Latent Diffusion

Formula (Autoencoder reconstruction loss):

FORMULA

$$L = \|x - \text{decoder}(\text{encoder}(x))\|^2$$

Formula (VAE — Evidence Lower Bound):

FORMULA

$$L = E[\log p(x|z)] - KL(q(z|x) \parallel p(z))$$

Formula (GAN minimax objective):

FORMULA

$$\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$$

Formula (Diffusion forward process):

FORMULA

$$q(x_t|x_{t-1}) = N(x_t; \sqrt{(1-\beta_t)} \cdot x_{t-1}, \beta_t I)$$

Code:

PYTHON

```
import torch.nn as nn

# Minimal Autoencoder
class Autoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Linear(784, 32)
        self.decoder = nn.Linear(32, 784)

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z)

# Using a pretrained diffusion model
from diffusers import StableDiffusionPipeline
pipe = StableDiffusionPipeline.from_pretrained("runwayml/stable-diffusion-v1-5")
image = pipe("a photo of a mountain lake").images[0]
```

Definition: Neural networks that operate directly on graph-structured data by propagating and aggregating information between connected nodes.

Purpose: learn representations for nodes, edges, or whole graphs

Use Cases: social networks, drug discovery, recommendation systems

Algorithms: GCN, GraphSAGE, GAT, GIN, Graph Transformer

Formula (Graph Convolution propagation rule):

FORMULA

$$H^{(l+1)} = \sigma(\tilde{D}^{(-1/2)} \tilde{A} \tilde{D}^{(-1/2)} H^{(l)} W^{(l)})$$

Code:

PYTHON

```
import torch
from torch_geometric.nn import GCNConv

class GCN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = GCNConv(in_channels=16, out_channels=32)

    def forward(self, x, edge_index):
        return self.conv1(x, edge_index)
```

Definition: Techniques for modeling and predicting future values from historical, time-ordered observations.

Purpose: forecast future values from past temporal patterns

Use Cases: weather, finance, demand forecasting

Algorithms: ARIMA, SARIMA, Prophet, LSTM, Transformer, N-BEATS, Temporal Fusion Transformer

Formula (ARIMA(p,d,q)):

FORMULA

$$(1 - \sum \phi_i L^i)(1-L)^d X_t = (1 + \sum \theta_i L^i) \varepsilon_t$$

p = autoregressive order, d = differencing order, q = moving-average order

Code:

PYTHON

```
from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(series, order=(1, 1, 1))
fit = model.fit()
forecast = fit.forecast(steps=10)

# Prophet
from prophet import Prophet
m = Prophet()
m.fit(df) # df with columns 'ds' and 'y'
future = m.make_future_dataframe(periods=30)
forecast = m.predict(future)
```

Definition: Systems that predict user preferences and suggest relevant items based on past behavior or similarity.

Purpose: personalize content/product suggestions

Use Cases: Netflix, Amazon, YouTube, Spotify

Algorithms: Collaborative Filtering, Content-Based Filtering, Matrix Factorization, Neural Collaborative Filtering, Two-Tower Models

Formula (Matrix Factorization):

FORMULA

$R \approx U \cdot V^T$ (R: user-item matrix, U: user factors, V: item factors)

Formula (Cosine similarity, for collaborative filtering):

FORMULA

$\text{sim}(A,B) = (A \cdot B) / (\|A\| \|B\|)$

Code:

PYTHON

```
from surprise import SVD, Dataset, Reader

reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(df[["user", "item", "rating"]], reader)
trainset = data.build_full_trainset()

model = SVD()
model.fit(trainset)
```

18

Anomaly Detection

Definition: Techniques that identify data points which deviate significantly from the norm.

Purpose: flag rare, suspicious, or abnormal observations

Use Cases: fraud detection, intrusion detection, manufacturing

Algorithms: Isolation Forest, One-Class SVM, Local Outlier Factor (LOF), Autoencoder, DBSCAN

Formula (Isolation Forest anomaly score):

FORMULA

$$s(x,n) = 2^{(-E[h(x)]/c(n))} \quad (\text{shorter avg. path length } h(x) \rightarrow \text{more anomalous})$$

Formula (Local Outlier Factor):

FORMULA

$$LOF(x) = \text{avg}(\text{lrd}(\text{neighbors of } x)) / \text{lrd}(x)$$

Code:

PYTHON

```
from sklearn.ensemble import IsolationForest

model = IsolationForest(contamination=0.05)
model.fit(X)
anomaly_scores = model.predict(X) # -1 = anomaly, 1 = normal
```

19

Ensemble Learning

Definition: Combines predictions from multiple models to achieve better performance than any single model.

Purpose: reduce variance/bias and boost predictive accuracy

Use Cases: high-accuracy predictive models, Kaggle competitions

Algorithms: Bagging, Boosting, Stacking, Voting, Blending

Formula (Bagging — averaged prediction):

FORMULA

$$\hat{y} = (1/M) \sum_m f_m(x)$$

Formula (Boosting — additive weighted models):

FORMULA

$$F(x) = \sum_m \alpha_m \cdot h_m(x)$$

Code:

PYTHON

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

model = VotingClassifier(estimators=[
    ("lr", LogisticRegression()),
    ("dt", DecisionTreeClassifier())
], voting="soft")
model.fit(X_train, y_train)
```

20 Optimisation Algorithms (Deep Learning)

Definition: Algorithms that iteratively update model parameters to minimize a loss function.

Purpose: train neural networks efficiently

Use Cases: training neural networks efficiently, large-scale language models

Optimizer	Formula
Batch Gradient Descent	$\theta \leftarrow \theta - \eta \nabla J(\theta)$ (full dataset each step)
Stochastic GD (SGD)	$\theta \leftarrow \theta - \eta \nabla J(\theta; x_i, y_i)$ (one sample at a time)
Mini-Batch GD	$\theta \leftarrow \theta - \eta \nabla J(\theta; \text{batch})$
Momentum	$v \leftarrow \beta v + (1-\beta) \nabla J(\theta); \theta \leftarrow \theta - \eta v$
Nesterov (NAG)	$v \leftarrow \beta v + \eta \nabla J(\theta - \beta v); \theta \leftarrow \theta - \eta v$
AdaGrad	$\theta \leftarrow \theta - (\eta / \sqrt{G + \epsilon}) \nabla J(\theta)$ (G = sum of squared past gradients)
RMSProp	$E[g^2] \leftarrow \beta E[g^2] + (1-\beta)g^2; \theta \leftarrow \theta - (\eta / \sqrt{E[g^2] + \epsilon})g$
AdaDelta	Like RMSProp, but replaces η with a running average of past updates
Adam	Combines Momentum + RMSProp with bias-corrected moment estimates
AdamW	Adam with decoupled weight decay
AdaMax	Adam variant using the infinity norm
Nadam	Adam + Nesterov momentum
Lion	Sign-based update: $\theta \leftarrow \theta - \eta \cdot \text{sign}(m)$
LAMB	Layer-wise adaptive learning rates for large-batch training

Formula (Adam — the most widely used):

FORMULA

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1-\beta_1)g_t \\v_t &= \beta_2 v_{t-1} + (1-\beta_2)g_t^2 \\ \hat{m}_t &= m_t / (1-\beta_1^t), \quad \hat{v}_t = v_t / (1-\beta_2^t) \\ \theta &\leftarrow \theta - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)\end{aligned}$$

Code:

PYTHON

```
import torch.optim as optim

optimizer_adam = optim.Adam(model.parameters(), lr=1e-3)
optimizer_adamw = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
optimizer_sgd = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```


Definition: Functions that quantify the difference between predicted and actual values, guiding optimization.

Purpose: define what "good" predictions mean for a given task

Use Cases: selecting the correct training objective for a task

Some Loss Functions

Category	Function	Formula
Regression	MSE	$(1/n) \sum (y - \hat{y})^2$
Regression	MAE	$(1/n) \sum y - \hat{y} $
Regression	Huber Loss	$\frac{1}{2}(y - \hat{y})^2$ if $ y - \hat{y} \leq \delta$
Regression	Log-Cosh	$\sum \log(\cosh(\hat{y} - y))$
Classification	Binary Cross-Entropy	$-\left[y \cdot \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$
Classification	Categorical Cross-Entropy	$-\sum y_i \log(\hat{y}_i)$
Classification	Sparse Cross-Entropy	Same as CCE with integer (not one-hot) labels
Classification	Focal Loss	$-\alpha(1 - \hat{y})^\gamma \log(\hat{y})$ (down-weights easy examples)
Classification	Hinge Loss	$\max(0, 1 - y \cdot \hat{y})$
Metric Learning	Contrastive Loss	$(1 - y) \frac{1}{2} D^2 + y \frac{1}{2} \max(0, m - D)^2$
Metric Learning	Triplet Loss	$\max(0, d(a, p) - d(a, n) + \text{margin})$
Metric Learning	Cosine Embedding Loss	$1 - \cos(x_1, x_2)$ for similar pairs
Generative	KL Divergence	$\sum p(x) \log(p(x)/q(x))$
Generative	Wasserstein Loss	$E[D(x_{\text{real}})] - E[D(x_{\text{fake}})]$

REFERENCE

Code:

PYTHON

```
import torch.nn as nn

mse = nn.MSELoss()
mae = nn.L1Loss()
huber = nn.HuberLoss()
bce = nn.BCELoss()
cce = nn.CrossEntropyLoss()
triplet = nn.TripletMarginLoss(margin=1.0)
```